



Darshan
UNIVERSITY

Python Programming - 2301CS404

Lab - 15 (1)

Roll No.: 514

Name: Meet Indoriya

In []:

01) Write a Program to create a class by name Students, and initialize attributes like name, age, and grade while creating an object.

In [8]:

```
class Student:
    def __init__(self , name , age , grade):
        self.name = name
        self.age = age
        self.grade = grade

    def display(self):
        print(f"Name: {self.name} Age: {self.age} Grade: {self.grade}")
s1 = Student("Meet" , 20 , "A")
s1.display()
```

Name: Meet Age: 20 Grade: A

02) Create a class named Bank_Account with Account_No, User_Name, Email,Account_Type and Account_Balance data members. Also create a method GetAccountDetails() and DisplayAccountDetails(). Create main method to demonstrate the Bank_Account class.

In []:

In [12]:

```
class bank_account:
    def __init__(self , username , email , account_type , account_balance):
        self.username = username
        self.email = email
        self.account_type = account_type
```

```

        self.account_balance = account_balance

    def GetAccountDetails(self):
        print(f"username: {self.username} \nEmail: {self.email} \nAccount Type:
b1 = bank_account("Meet" , "meetindoriya@gmail.com" , "Saving" , 1000000000000)
b1.GetAccountDetails()

```

username: Meet
Email: meetindoriya@gmail.com
Account Type: Saving
Balance: 1000000000000

03) WAP to create Circle class with area and perimeter function to find area and perimeter of circle.

```

In [24]: class circle:

    def __init__(self , radius):
        self.radius = radius
    def area(self):
        print(f"Area: {3.14 * self.radius * self.radius}")

    def perimeter(self):
        print(f"Perimeter: {2 * 3.14 * self.radius : .2f}")

c1 = circle(10)
c1.area()
c1.perimeter()

```

Area: 314.0
Perimeter: 62.80

04) Create a class for employees that includes attributes such as name, age, salary, and methods to update and display employee information.

```

In [28]: class employee:
    def __init__(self , name , age , salary):
        self.name = name
        self.age = age
        self.salary = salary

    def employeeDisplay(self):
        print(f"Name = {self.name} Age: {self.age} Saalry: {self.salary}")

    def updateSalary(self):
        salary = int(input("Enter The Salary To Update"))
        increment = self.salary + salary
        print(f"Updated Salary {increment :.2f}")
e1 = employee("Meet", 20 , 5000)
e1.employeeDisplay()
e1.updateSalary()

```

Name = Meet Age: 20 Saalry: 5000
Updated Salary 5010.00

05) Create a bank account class with methods to deposit, withdraw, and check balance.

```
In [37]: class bank:
    Balance = 10000
    def deposit(self):
        amount = int(input("Enter The Amount To Deposit: "))
        self.Balance -= amount
    def withdraw(self):
        amount = int(input("Enter The Amount To Withdraw: "))
        self.Balance += amount
    def checkBalance(self):
        print(f"Account Balance: {self.Balance}")

b1 = bank()
b1.deposit()
b1.checkBalance()
b1.withdraw()
b1.checkBalance()
```

Account Balance: 9949
Account Balance: 9961

06) Create a class for managing inventory that includes attributes such as item name, price, quantity, and methods to add, remove, and update items.

```
In [47]: class inventory:

    def __init__(self):
        self.name = input("Enter The Name Of Product: ")
        self.price = int(input("Enter The Price Of Product: "))
        self.quantity = int(input("Enter The quantity of Product: "))
        print(f"{self.name} \nPrice: {self.price}")
    def add(self):
        self.newName = input("Enter The Product Name: ")
        print("Product Added")

    def remove(self):
        self.remove = input("Enter The Name To Remove Product")
        print("Product Remove")

    def update(self):
        self.update = input("Enter The Name Of Product To Update")
        print("Product Update")
i1= inventory()
i1.add()
```

Product Added

07) Create a Class with instance attributes of your choice.

```
In [38]: class university:
    name = "Darshan University"
    location = "Rajkot"
    course = "CSE"
```

08) Create one class student_kit

Within the student_kit class create one class attribute principal name (Mr ABC)

Create one attendance method and take input as number of days.

While creating student take input their name .

Create one certificate for each student by taking input of number of days present in class.

```
In [45]: class student_kit:
    principle_name = "Mr. ABC"

    def __init__(self):
        self.name = input("Name: ")

    def attendance(self):
        self.attendDay = int(input("Enter The Number Of Days Student Present: "))
        self.attendDay = (self.attendDay / 100) * 100

    def certificate(self):
        print(f"Name: {self.name} \nAttendance: {self.attendDay : .2f}%")
s2 = student_kit()
s2.attendance()
s2.certificate()
```

Name: Meet

Attendance: 99.00

09) Define Time class with hour and minute as data member. Also define addition method to add two time objects.

```
In [2]: class Time:
    def __init__(self, hour, minute):
        self.hour = hour
        self.minute = minute

    def Addition(self, other):
        total_minutes = self.minute + other.minute
        total_hours = self.hour + other.hour

        if total_minutes >= 60:
            total_hours += total_minutes // 60
            total_minutes = total_minutes % 60

        return Time(total_hours, total_minutes)

    def display(self):
        print(f"{self.hour} hour(s) and {self.minute} minute(s)")

T1 = Time(2, 45)
T2 = Time(1, 30)
T3 = T1.Addition(T2)

T1.display()
```

```
T2.display()  
T3.display()
```

```
2 hour(s) and 45 minute(s)  
1 hour(s) and 30 minute(s)  
4 hour(s) and 15 minute(s)
```

In []: